

# pascalStirlingQuad — Module Specification

**Purpose (one line)** A compact, input-agnostic read/write module that analyzes small numeric patches using Pascal (binomial) and Stirling-inspired operations and encodes/decodes compact local representations for downstream pipelines.

---

## Design goals

- **Encapsulated:** single-file/module boundary with a clean API.
  - **Input-agnostic:** accepts 1D, 2D (H×W, H×W×C), or 1D attribute arrays; works on numeric tensors.
  - **Read/Write enabled:** produces serializable blobs and reconstructs patches with optional residuals.
  - **Interpretable:** outputs are Newton/binomial coefficients, difference-energy summaries, and quadtree metadata.
  - **Lightweight:** minimal dependencies (NumPy-style array ops, optional compressor).
- 

## High-level capabilities

- Separable Pascal smoothing (multiple scales)
  - Forward differences ( $\Delta$ ,  $\Delta^2$ ,  $\Delta^3$ ) and detail maps
  - Per-quadrant Pascal pyramids and energy scoring
  - Adaptive quadtree splitting guided by Stirling-informed energies
  - Encoding leaf nodes as Newton/binomial coefficients
  - Optional storage of compressed residuals
  - Serialization / deserialization (JSON + binary arrays)
- 

## Core data structures

**PatchAnalysis** (returned by `analyze_patch()`)

```
{
  "shape": [H, W, C],
  "pascal_pyramids": {"quadrant_id": [arrays...]},
  "difference_maps": {"order": array},
  "quadrant_energy": {"q0": float, ...},
  "polynomial_order_estimate": int,
  "stirling_coeffs": [[...]],
  "partition_candidates": [ {"mask": ..., "score": ...}, ... ]
}
```

## PatchBlob (returned by `encode_patch`)

```
{
  "header": {"type": "psq", "shape": [H, W, C], "dtype": "float32", "created": [...]},
  "quadtree_meta": [
    {"x": int, "y": int, "w": int, "h": int, "coeff_index": int} , [...] ],
  "coefficients": [ [[coeffs per channel]], [...] ],
  "energy_stats": [float, [...] ],
  "residuals": optional binary blob (compressed)
}
```

## Public API (concise signatures)

- `analyze_patch(patch: ndarray, params: dict) -> PatchAnalysis`
- `encode_patch(analysis: PatchAnalysis, params: dict) -> PatchBlob`
- `reconstruct_patch(blob: PatchBlob, params: dict) -> ndarray`
- `serialize_blob(blob: PatchBlob) -> bytes`
- `deserialize_blob(bytes) -> PatchBlob`
- `compute_pascal_kernel(length: int) -> ndarray`
- `separable_convolve(patch, kernel_x, kernel_y) -> ndarray`
- `compute_forward_differences(patch, order: int) -> ndarray`
- `quadtree_split(patch, energy_threshold: float, max_depth: int) -> quadtree_meta`
- Utility getters: `patch_rmse(orig, recon)`, `visualize_quadtree(blob)`

## Parameters (recommended defaults)

- `pascal_scales = [3, 5, 7]` — 1D kernel lengths
- `difference_orders = [1, 2]`
- `energy_threshold = 0.02` (normalized)
- `max_quadtree_depth = 5`
- `residual_threshold = 0.01` (RMSE)
- `channel_mode = 'per-channel' or 'luminance'`
- `compressor = None or {'method': 'zlib', 'level': 6}`

## Processing flow (read path)

1. **Accept** patch and basic params. Normalize if requested.
2. **Per-channel** or luminance conversion.
3. **Compute Pascal pyramids**: for each quadrant and scale, apply separable Pascal kernels (rows then cols).
4. **Compute differences**:  $\Delta$ ,  $\Delta^2$  (per axis and combined) on smoothed outputs.
5. **Score energy**: per-quadrant energy = weighted mean( $|\Delta^k|$ ) + variance.

6. **Quadtree split**: recursively split quadrants whose `energy > threshold` until `max_depth` or `low energy`.
  7. **Encode leaves**: compute Newton/binomial coefficients (from forward differences) for leaf tiles; optionally compute and compress residuals.
  8. **Package** PatchBlob with metadata, coefficients and optional residuals.
- 

## Reconstruction flow (write path)

1. Deserialize blob.
  2. For each leaf, expand Newton/binomial coefficients to tile pixels.
  3. Add residuals if present.
  4. Stitch tiles back into patch (blend overlaps if used).
  5. Return full reconstructed patch.
- 

## Integration points (with Schur & other pipelines)

- **Pre-triangle smoothing**: run on texture maps or 2D parameterizations before triangulation.
  - **Adaptive meshing**: map per-tile energy to local triangle density / refine criteria.
  - **Attribute streaming**: store vertex attributes as compact coefficients for streaming and decode on client.
  - **Segmentation-driven remeshing**: use partition candidates to seed seams and region-based remesh.
- 

## Performance notes

- Separable convolutions are  $O(H \cdot W \cdot S)$  and cheap in practice.
  - Differences are  $O(H \cdot W \cdot K)$ . Quadtree adapts work—smooth regions stop early.
  - Coefficients per leaf are small (degree  $\leq 3 \rightarrow \sim 4$  numbers/channel).
  - Parallelize across patches/nodes for throughput.
- 

## Serialization

- Metadata as JSON; numeric arrays stored as binary (NumPy `.npy`) or appended bytes.
  - Optionally produce a single `.psq` (zip) containing metadata + binary arrays.
- 

## Example usage (pseudocode)

```
analysis = analyze_patch(patch, params)
blob = encode_patch(analysis, params)
bytes_blob = serialize_blob(blob)
# save bytes_blob
# later
```

```
blob = deserialize_blob(bytes_blob)
recon = reconstruct_patch(blob, params)
```

---

## Testing checklist

- Unit tests for Pascal kernel generation and symmetry.
  - Finite-difference tests on polynomials ( $\Delta^k$  matches Stirling-scaled constants).
  - Quadtree roundtrip: analyze  $\rightarrow$  encode  $\rightarrow$  reconstruct  $\rightarrow$  RMSE  $\leq$  residual\_threshold.
  - Integration smoke test with Schur: use tile energies to influence local subdivision.
- 

## Implementation notes & tips

- Keep coefficient degrees small ( $\leq 3$  or  $\leq 4$ ) for compactness.
  - Use overlapping tiles and blending at stitch to reduce boundary artifacts.
  - Allow `encode_patch` to store raw residuals only when necessary to meet fidelity.
  - Keep module dependency-free other than a NumPy-like numeric package; provide optional fast C/Numba paths.
- 

*End of spec — concise, implementable, and ready to drop into a pipeline.*